Лабораторная работа №4
Создание лендинга с использованием современных технологий искусственного интеллекта

Цель: получение практических навыков применения современных технологий искусственного интеллекта для разработки web-сайтов.

Теоретические сведения
Большая языковая модель

Одним из активно развивающихся направлений искусственного интеллекта является **генеративный искусственный интеллект (Generative AI)**.

В рамках данной лабораторной работы рассматривается применение генеративного ИИ для разработки программного кода web-сайта.

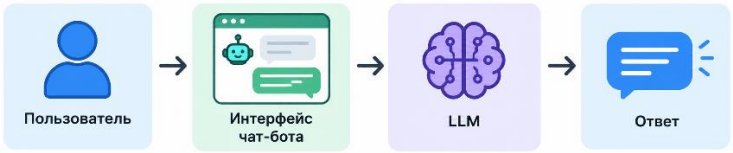
Большая языковая модель (Large Language Model, LLM) – это нейросетевая модель, предназначенная для обработки и генерации естественного языка на основе закономерностей, изученных в процессе обучения на больших объёмах текстовых данных.

Современные LLM способны выполнять широкий спектр задач:

- отвечать на вопросы;
- создавать и редактировать тексты;
- переводить тексты;
- обобщать информацию;
- анализировать документы;
- объяснять программный код;
- генерировать программный код;
- исправлять ошибки в коде;
- преобразовывать текст из одного формата в другой;
- создавать инструкции и техническую документацию.

Важно понимать, что **LLM и чат-бот** – не одно и то же.

- LLM – модель искусственного интеллекта
- Чат-бот – программный интерфейс, предоставляющий пользователю возможность взаимодействовать с моделью посредством диалога.



Популярные большие языковые модели (LLM):

- GigaChat (Сбер)
- YandexGPT (Яндекс)
- Gemini (Google)
- GPT (OpenAI)
- Claude (Anthropic)
- Grok (xAI)
- Deepseek (DeepSeek AI).
- Kimi (Moonshot AI).
- Qwen (Alibaba Group).

Контекст и контекстное окно

Контекст – это информация, доступная модели при формировании очередного ответа. В контекст могут входить:

- сообщения пользователя;
- предыдущие сообщения модели;
- текстовые документы;
- фрагменты программного кода;
- результаты предыдущих действий;
- дополнительные инструкции;
- изображения и другие данные, если система их поддерживает.

У LLM существует ограниченный объём информации, который может одновременно учитываться при генерации ответа. Этот объём называется **контекстным окном**.

Контекстное окно измеряется в токенах.

Токен – элемент текста, используемый моделью для обработки входной и выходной информации.

Советы:

- не смешивать в одном диалоге большое количество независимых задач;
- предоставлять модели только необходимую информацию;
- структурировать контекст;
- явно описывать назначение передаваемого кода;
- разделять обсуждение разных проектов или задач.

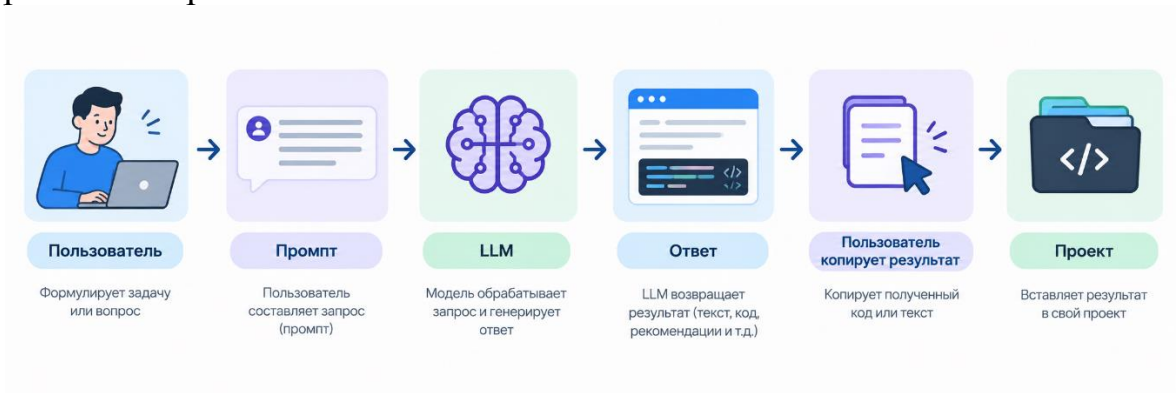
Чат-бот на основе LLM – программная система, предоставляющая пользователю диалоговый интерфейс для взаимодействия с большой языковой моделью.

Пользователь формулирует запрос (промт), модель анализирует доступный контекст и генерирует ответ.

Например:

«Создай современный одностраничный сайт кофейни с адаптивным дизайном».

LLM может вернуть код, который пользователь затем самостоятельно переносит в проект.



Если необходимо изменить сайт, пользователь должен:

1. найти нужный файл;
2. скопировать код;
3. передать его модели;
4. сформулировать задачу;
5. получить исправленный вариант;
6. самостоятельно заменить код в проекте;
7. проверить результат.

Таким образом, значительная часть работы остаётся на стороне человека.

Промпт и его структура

Промпт – текстовая инструкция, предоставляемая пользователем искусственному интеллекту для определения требуемого результата.

Качество результата LLM существенно зависит от того, насколько точно сформулирована задача и насколько качественный контекст предоставлен модели.

Простой запрос:

«*Сделай сайт для кофейни*».

является недостаточно информативным.

Модель **не знает**:

- какая целевая аудитория;
- какой визуальный стиль требуется;

- какая структура сайта;
- какие технологии необходимо использовать (стек);
- какие функции должны присутствовать;
- каким должен быть результат;
- какие ограничения существуют.

Один из вариантов структурирования запроса – модель **R-T-F-C**:

- **R – Role** – роль;
- **T – Task** –задача;
- **F –Format** – формат результата;
- **C – Constraints** – ограничения.

В расширенном варианте рекомендуется также явно выделять **контекст**. Хотя иногда под контекстом понимаются все эти части (и технологии, и задача, и результаты).

Часто важен не сам промт, а правильный набор контекста: чем качественнее определён контекст, тем меньше необходимости модели самостоятельно придумывать отсутствующую информацию.

1. **Role** – роль

Определяется роль, в которой должна действовать модель.

Например:

«Ты опытный frontend-разработчик, специализирующийся на создании современных адаптивных лендингов».

Роль позволяет задать необходимый уровень и направление экспертизы модели.

Можно уже на этом этапе добавить стек технологий:

«Ты опытный frontend-разработчик с 8-летним опытом в создании лендингов для малого бизнеса. Ты специализируешься на React 18 + TypeScript, знаешь принципы UX/UI и следишь за трендами веб-дизайна 2026 года.»

2. **Context** – контекст

Описывается предметная область и исходная ситуация.

Например:

«Заказчик – уютная тематическая кофейня «Кофейный котик» в Москве. Целевая аудитория: молодые люди, студенты и зумеры, которые ценят теплую дружелюбную атмосферу, милую кошачью эстетику, приходят пообщаться, поучиться с ноутбуком или взять кофе на вынос. У бренда есть: меню, фотографии напитков. Важно использовать светлую, мягкую теплую палитру (молочный, бежевый, карамельный, кофейный цвета)».

3. **Task** –задача

Чёткое указание, что нужно сделать, какие задачи этот код должен решать.

Например:

«Создай одностраничный адаптивный лендинг». – слишком мало контекста.

Желательно описывать структуру результата:

- главный экран;
- информация о компании;
- преимущества;
- каталог;
- отзывы;
- контакты;
- форма обратной связи;
- подвал сайта.

Например,

«Создай современный, эстетичный и полностью адаптивный одностраничный лендинг со следующей структурой:

1. Header / Навигация: логотип «Кофейный котик», ссылки на разделы (О нас, Меню, Акции, Бонусы, Контакты) с эффектом залипания (sticky).

2. Hero-секция: запоминающийся слоган («Мурчим от счастья с каждой чашкой»).

3. Раздел «О нас»: теплая история создания кофейни (тактильность, забота о гостях, кошачий уют).

4. Раздел «Фирменное меню»: демонстрация 6 популярных напитков (например: Эспрессо «Мурзик» и т.д.) с ценами, карточками товаров.

5. Раздел «Акции и Бонусная программа»: плашки с текущими предложениями и правилами начисления бонусов лояльности.

6. Раздел «Контакты и график»: адрес в Москве, часы работы, карты и ссылки на соцсети.»

4. **Format** – формат результата.

Ожидаемый результат – в каком виде должен быть ответ (таблица, код, список, текст).

Например:

«Предоставь полный HTML-файл, содержащий HTML и CSS в одном файле».

или:

«Предоставь структуру проекта и полный код каждого файла с указанием имени файла».

5. **Constraints** – ограничения.

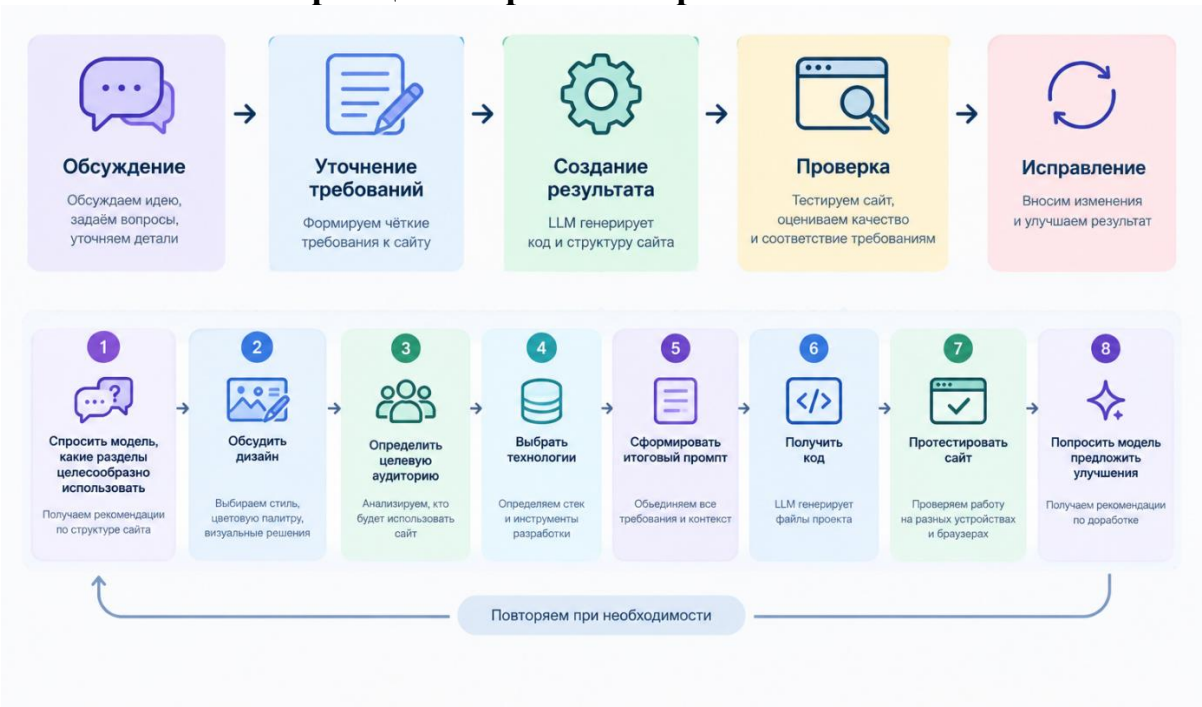
Указываются ограничения, которым должен соответствовать результат.

Например:

- адаптивность;
- отсутствие сторонних библиотек;
- использование только HTML и CSS;
- поддержка мобильных устройств;
- определённая цветовая палитра;
- определённое количество разделов;
- отсутствие несуществующих библиотек;
- использование семантической разметки.

«Ограничения:
НЕ используй Bootstrap или другие CSS-фреймворки – только CSS.
НЕ подключай внешние шрифты с Google Fonts.
Совместимость: Chrome 110+, Safari 16+, мобильные браузеры 2024+.
Не добавляй несуществующие API или библиотеки – используй только стабильные релизы на сентябрь 2026.»

Принцип итеративной работы с LLM



LLM может использоваться не только как генератор конечного результата, но и как инструмент проектирования.

AI-агенты

Обычный чат-бот обычно не обладает полноценным доступом к структуре локального проекта пользователя.

AI-агент – программная система на основе искусственного интеллекта, способная не только генерировать ответы, но и самостоятельно выполнять последовательность действий для достижения поставленной цели с использованием доступных инструментов.

Ключевое отличие агента заключается в наличии **инструментов** и **возможности выполнять действия во внешней среде**.



AI-агент кодинга может:

- просматривать структуру проекта;
- читать файлы;
- создавать файлы;
- изменять существующий код;
- искать ошибки;
- запускать команды;
- выполнять тесты;
- анализировать результаты выполнения;
- повторять действия после обнаружения ошибки;
- рефакторить код.

Таким образом,

- LLM отвечает на запрос.
- Чат-бот организует взаимодействие пользователя с LLM.
- AI-агент использует LLM для выполнения задачи с помощью инструментов.

Примеры AI-агентов:

- Cursor – отдельный самостоятельный редактор (среда разработки).
- Google Antigravity – агентная среда разработки от Google.
- Cline – AI-агент для разработки, который может работать внутри среды разработки (например, в VS Code).
- OpenCode – открытый AI-агент, который используется через терминал, хотя есть и десктопная версия.
- Codex – AI-агент от OpenAI, можно использовать в терминале, в приложении или через интеграцию в среду разработки (расширение).
- Claude code – AI-инструмент от Anthropic, может использовать в терминале, а также в различных средах разработки (например, VS Code).
- Gemini CLI – инструмент командной строки для взаимодействия с моделями Gemini и выполнения агентских задач разработки.

– SourceCraft Code Assistant – AI-инструмент для разработчиков, интегрируемый непосредственно в среду разработки
– GigaCode – AI-инструмент для разработчиков, связанный с экосистемой СберТех.

Важно: условия доступа и использования различных AI-агентов могут существенно отличаться: некоторые инструменты предоставляют бесплатный доступ или ограниченное количество запросов, другие требуют оформления платной подписки или оплаты используемых моделей. Кроме того, доступные функции и установленные лимиты могут изменяться со временем.

AI-агент обладает большей автономностью, поэтому необходимо контролировать его действия.

AI-агент может:

- неправильно интерпретировать задачу;
- изменить не тот файл;
- удалить нужный код;
- использовать несуществующую библиотеку;
- предложить устаревший API;
- создать лишние зависимости;
- неправильно определить причину ошибки;
- выполнять слишком большое количество ненужных действий.

Поэтому важно:+

ИИ выполняет работу, человек контролирует результат.

Порядок выполнения лабораторной работы
Общие упражнения

Для начала предлагается выполнить ряд общих упражнений для более глубокого понимания, а затем перейти к самостоятельной работе. Также материалы данного раздела помогут выполнить индивидуальное задание. Для данного этапа **можно не предоставлять скриншотов** (либо предоставить несколько скриншотов, без подробного описания).

1. В работе предлагается использовать Visual Studio Code – среда для разработки кода (если у вас не установлена данная IDE* установите её: https://code.visualstudio.com/Download/).

* **IDE** (Integrated Development Environment или интегрированная среда разработки – это программный комплекс, предоставляющий программисту единое рабочее пространство со всеми необходимыми инструментами для разработки программного обеспечения (текстовый редактор кода, инструменты сборки и компиляции, отладчик и т.д.).

2. Создайте папку проекта. Откройте рабочую папку и создайте новый файл index.html – стандартный базовый файл для любого сайта (рис.1).

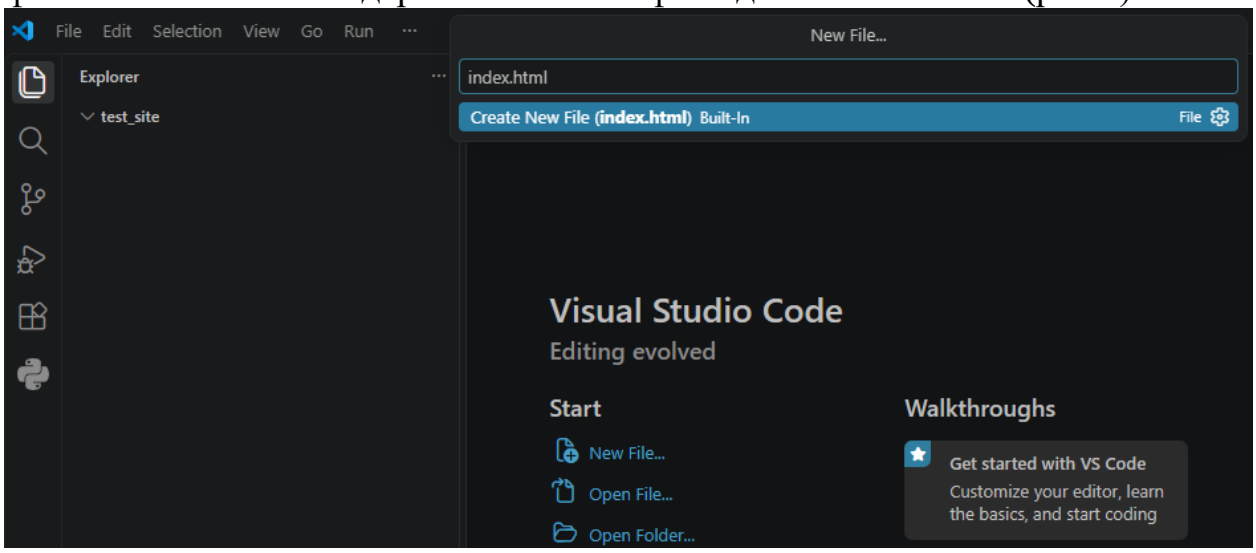


Рисунок 1 – Создание нового файла в рабочей папке test_site

2. Установить расширение Live Server, которое позволяет визуализировать код сайта (через Extensions -поиск - Install)

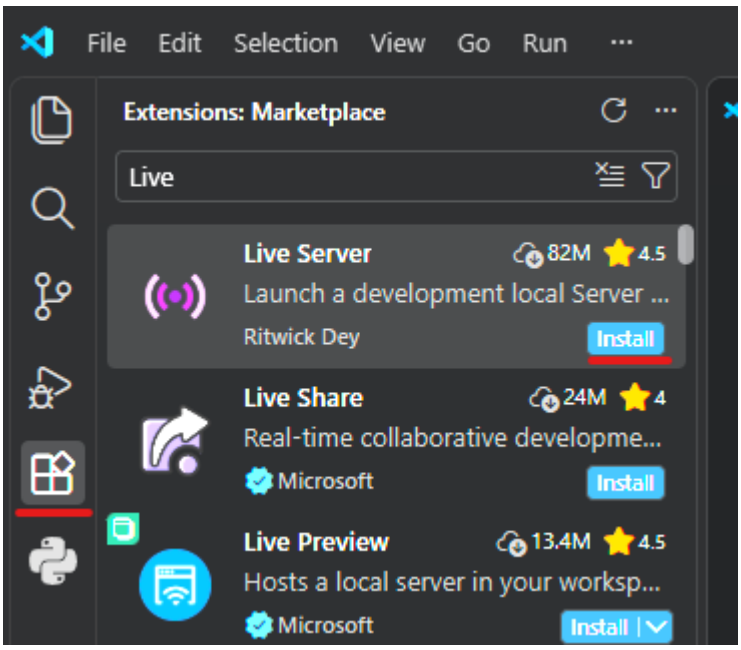


Рисунок 2 – Установка Live Server

3. Откроем чат-бот с LLM. Например, https://giga.chat/ – GigaChat (*вы можете выбрать другую LLM, при выполнении индивидуальной части* ***обязательно указать, что использовали***).

4. Используем следующий промт:

Создай современную, адаптивную лендинг-страницу для кофейни «Кофейный котик».

Требования:

Главный раздел с ярким слоганом и кнопкой призыва к действию

Навигационное меню, которое остается сверху при прокрутке

Раздел «О компании», объясняющий нашу историю

Раздел с предварительным просмотром меню, в котором представлены 6 фирменных напитков.

Раздел Акции, где представлено 2-3 актуальные акции

Раздел Бонусная программа, где описаны правила лояльности кофейни

Раздел контактов с указанием часов работы и местоположения

Используй теплую цветовую схему с бежевыми и коричневыми оттенками.

Сделай сайт удобным для мобильных устройств

Добавь плавную прокрутку между разделами

Включи современные CSS-эффекты, такие как анимация при наведении курсора

Предоставь, пожалуйста, полный HTML и CSS в одном файле

Задание для студентов: Изучите этот промт, оцените, из каких частей он состоит, соответствует ли формату **R-T-F-C.**

5. После того, как вы получите ответ от LLM со сгенерированным кодом необходимо его скопировать и вставить в файл index.html в VS Code.

6. Проверьте отображение сайта в браузере, для этого нажмите Go Live и откройте сайт в браузере (рис.3).

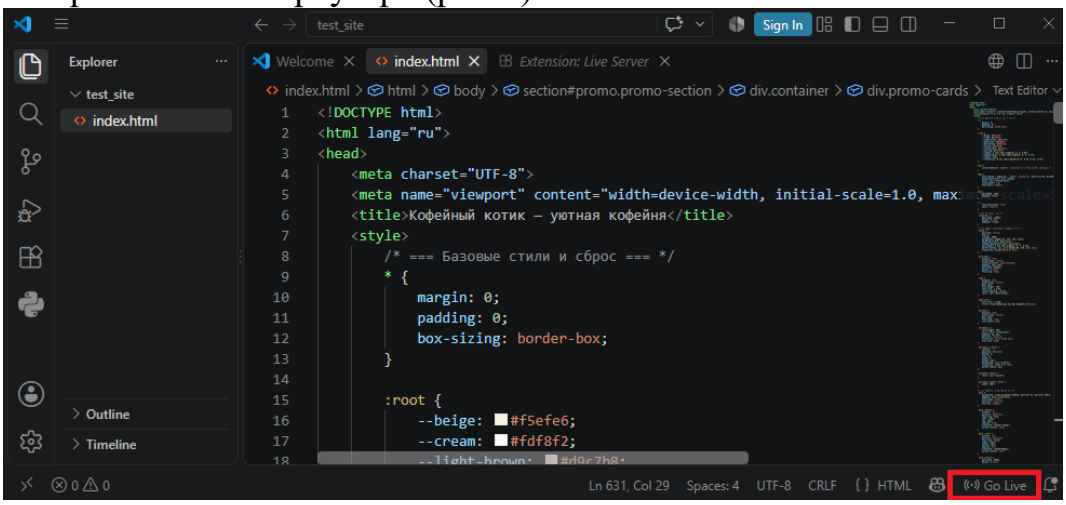


Рисунок 3 – Запуск Live Server

После запуска Live Server в VS Code сайт автоматически откроется в браузере. Убедитесь, что сайт отображается корректно: заголовки читаемы, изображения подгружаются, все блоки находятся на своих местах.

Проверьте работу ссылок, кнопок.

Как правило, при создании сайтов необходимо учитывать некоторые требования. Обычно сайт должен:

- корректно отображаться на компьютере;
- корректно отображаться на мобильном устройстве;
- иметь логичную структуру;
- использовать единый визуальный стиль;
- содержать рабочую навигацию;
- иметь кнопки и интерактивные элементы;
- использовать читаемые шрифты;
- иметь визуально выделенные заголовки;
- содержать адаптивную вёрстку.

Дополнительно рекомендуется использовать:

- плавную прокрутку;
- эффекты при наведении;
- анимацию появления;
- карточки;
- иконки;
- изображения;
- кнопки;
- адаптивное меню.

Как проверить **адаптивность** сайта:

Сжать окно браузера вручную. Просто потяните за правый или левый край окна и посмотрите, как элементы перестраиваются.

Использовать DevTools в браузере Chrome:

Правый клик по странице "Просмотр кода" или F12 – вкладка Toggle Device Toolbar (значок телефона сверху слева).

В верхней панели можно выбрать разные модели устройств (iPhone, Pixel и т.д.) и посмотреть отображение на мобильном устройстве.

Скорее всего на данном этапе при отображении сайта могут быть какие-то проблемы (не отображается картинка, какие-то кнопки не работают, нет анимации и т.д.). Сначала **не нужно** вносить никаких изменений в код, действуйте поэтапно.

7. На рис.4-6 представлены результаты сгенерированного сайта разными моделями. Для всех моделей использовался один промт. Обратите внимание на отличие дизайна у разных моделей.

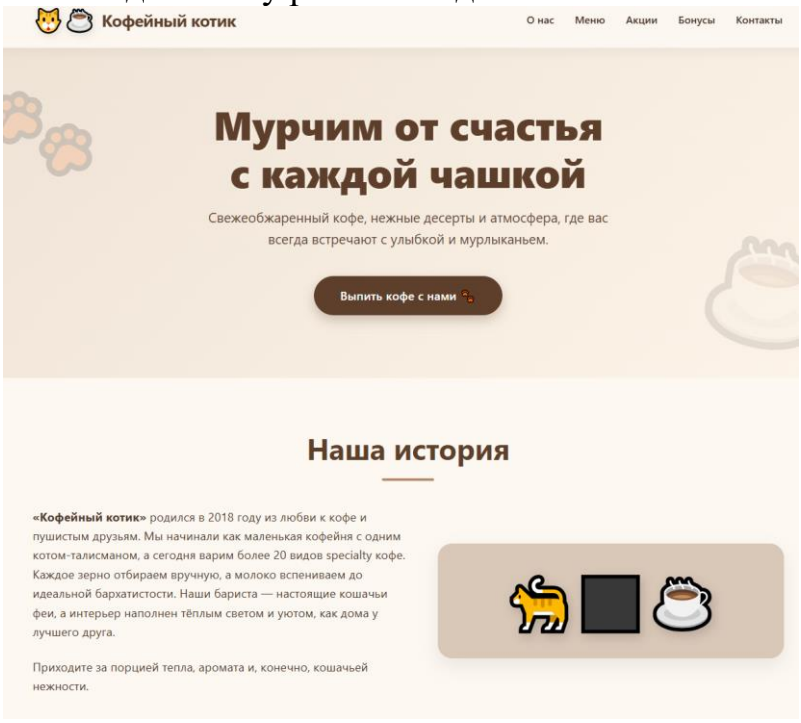


Рисунок 4 – Результат Deepseek

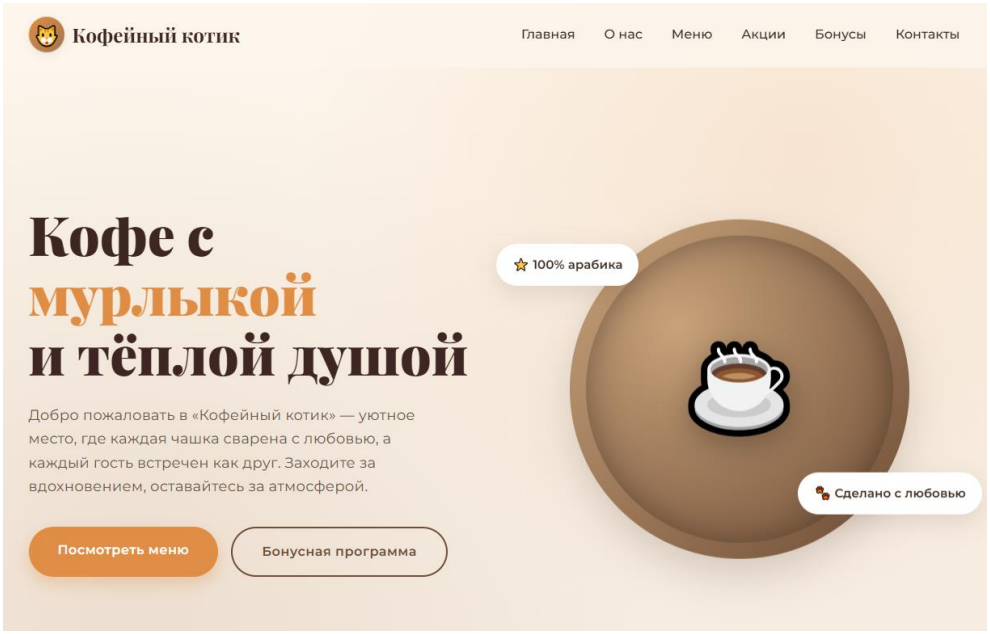


Рисунок 5 – Результат Qwen

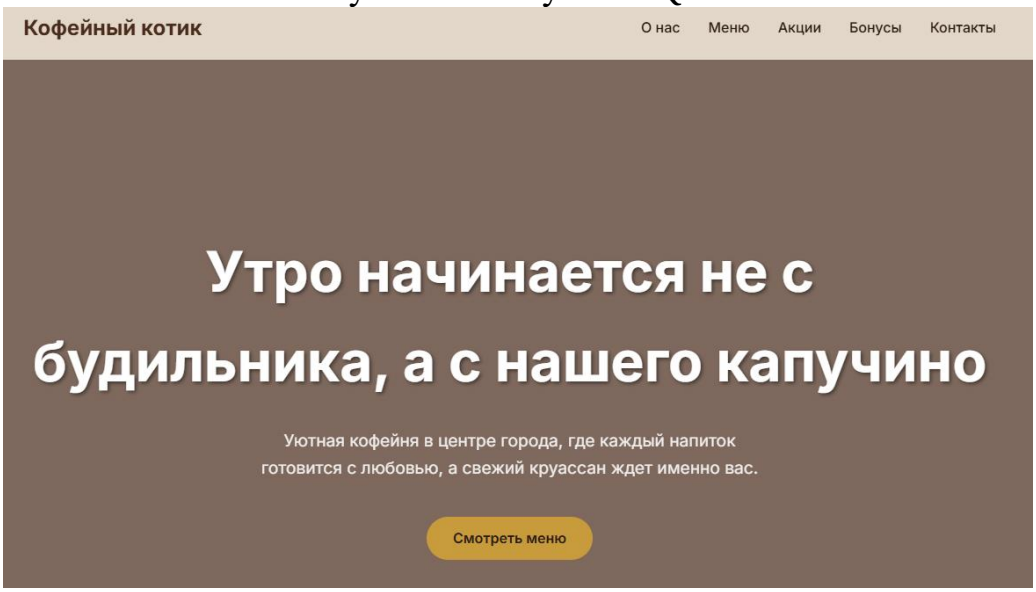


Рисунок 6 – Результат GigaChat

8. Видно, что дизайны сайтов отличаются, чтобы понять, хорошо ли выполнен дизайн безусловно важно иметь некоторую насмотренность, чтобы понимать, насколько действительно качественно сгенерирован контент с помощью ИИ.

Например, на рис. 7 показан блок с меню, где видно, что 6 напитков отображаются не очень эстетично, либо нужно оставить 5, либо вместить шесть в одну строку, либо разделить на две строки по три. Можно попросить модель исправить данный фрагмент, чтобы сделать его более наглядным (результат на рис. 8).

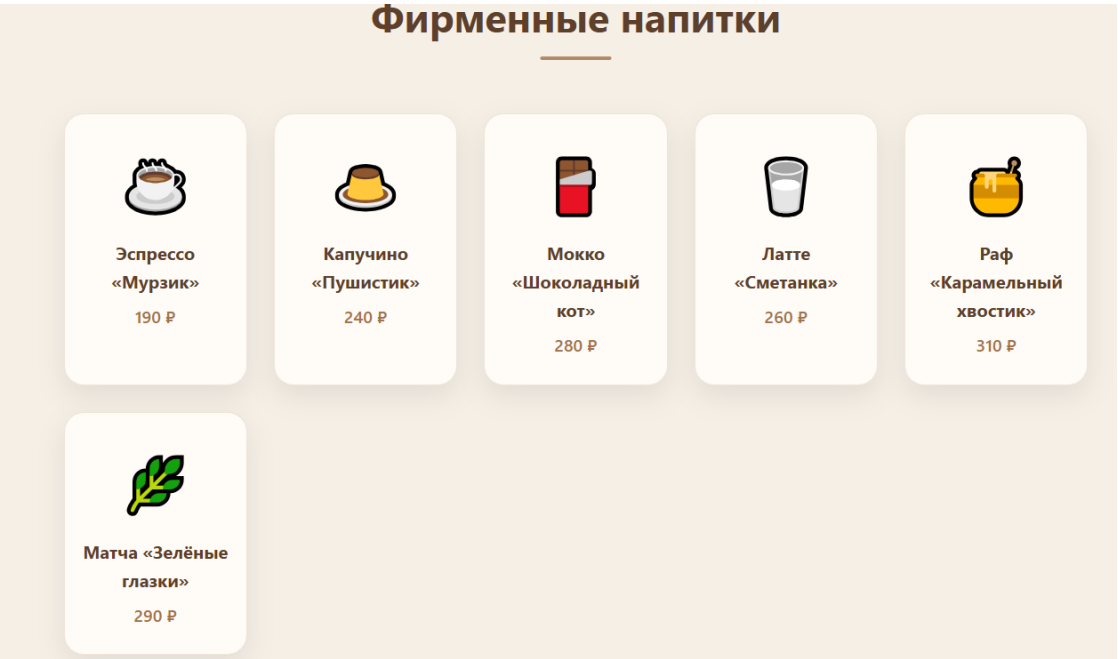


Рисунок 7 – Скриншот блока с меню

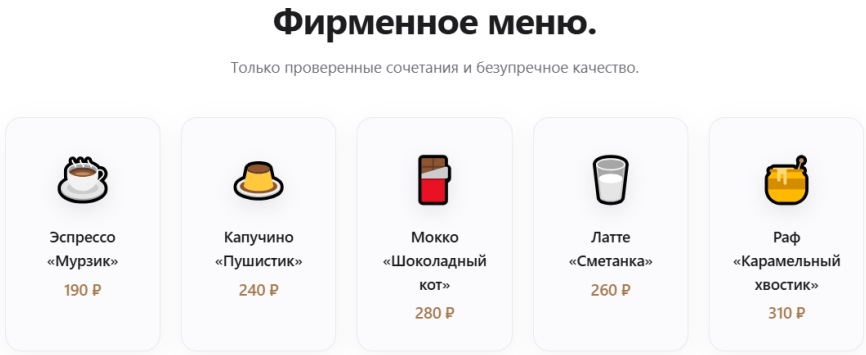


Рисунок 8 – Скриншот нового блока с меню

9. Использование LLM для проверки сайта.

Можно проверять работу ИИ с помощью LLM.

Если ваш сайт опубликован – можно предоставить в чат-бот ссылку (URL) и написать промт «*Проанализируй этот сайт, скажи, что в дизайне можно было улучшить, сделать интереснее. Найди ошибки, которые необходимо исправить. Дай советы по улучшению кода*». При этом сам промт можно также предварительно согласовать с LLM, чтобы наиболее четко сформулировать то, что необходимо проверить.

Также можно вставить код и выполнить его проверку, сделать скриншоты сайта или записать видео, если чат-бот поддерживает отправку изображений и видео.

Важно, что некоторый анализ с помощью LLM может давать ошибки, поэтому лучше просить выполнить анализ вашего сайта в новом окне чата и проверять результат на адекватность.

10. Теперь мы будем использовать AI-агента. В лабораторной работе предлагается использовать https://sourcecraft.dev (*необходимо создать*

аккаунт, чтобы получить доступ, при этом предоставляется бесплатный доступ с некоторыми ограничениями, квоты можно проверить в разделе Квота).

Source Craft Code Assistant – это AI-помощник разработчика, основанный на больших языковых моделях (LLM), с помощью которого вы можете улучшить процесс написания кода и повысить производительность.

Мы будем работать с этим агентом в VS Code. Можете ознакомиться с дополнительной информацией по ссылке: https://sourcecraft.dev/portal/docs/ru/code-assistant/?tabs=ide_vscode#install-plugin

Обратите внимание, что можно создавать репозитории непосредственно в браузерной версии (рис.9).

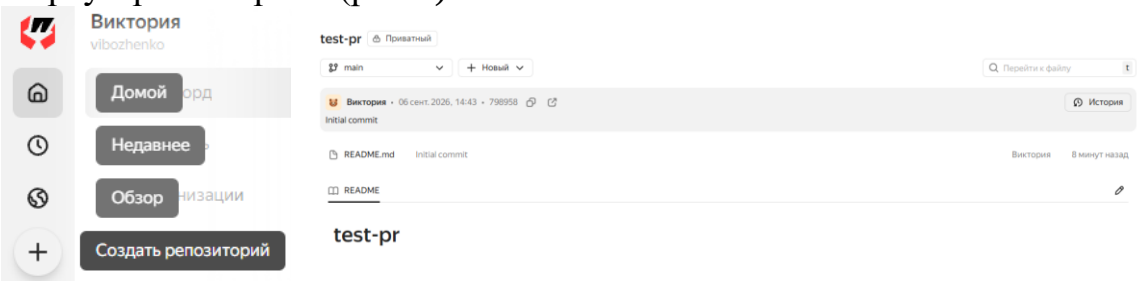


Рисунок 9 – Результат создания репозитория в Source Craft

В качестве эксперимента можно использовать Чат Code Assistant прямо в браузере и предложить ему тот же промт для создания кофейни, только не уточняя, что нам нужен один файл html. Фрагмент ответа на рис. 10. В результате работы были созданы файлы сайта и автоматически размещены в репозиторий проекта (рис.11).

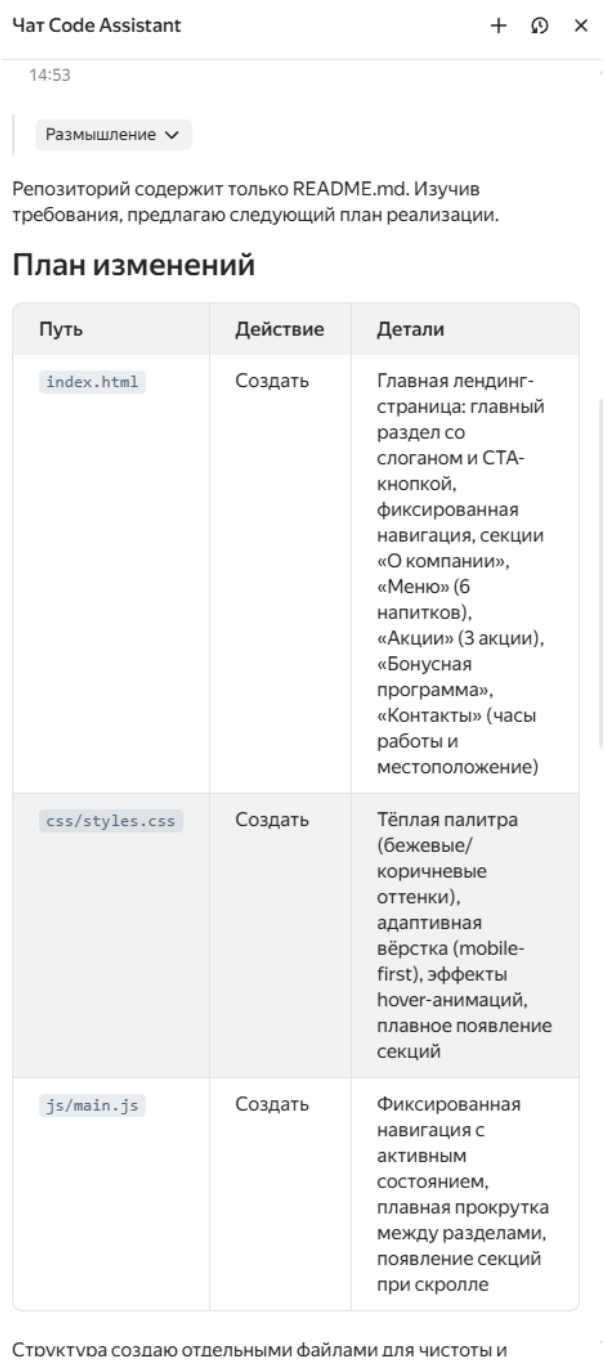


Рисунок 10 – Фрагмент работы агента в Source Craft

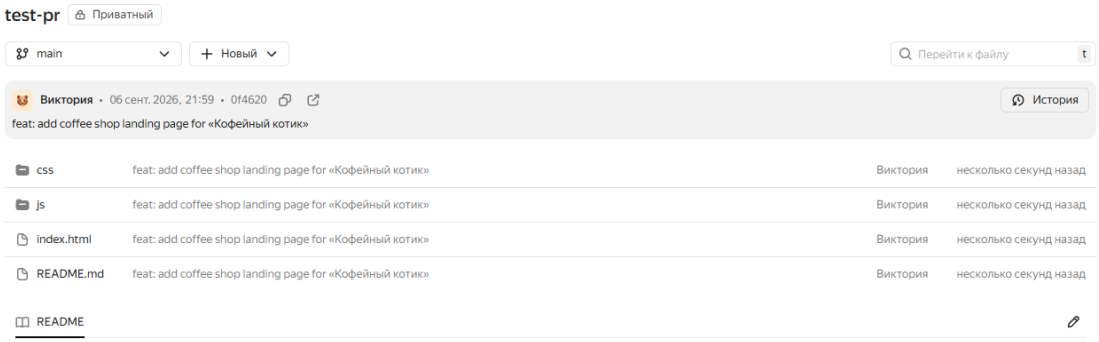


Рисунок 11 – Создание сайта в Source Craft

11. Установите расширение в VS Code – SourceCraft Code Assistant (рис. 12).

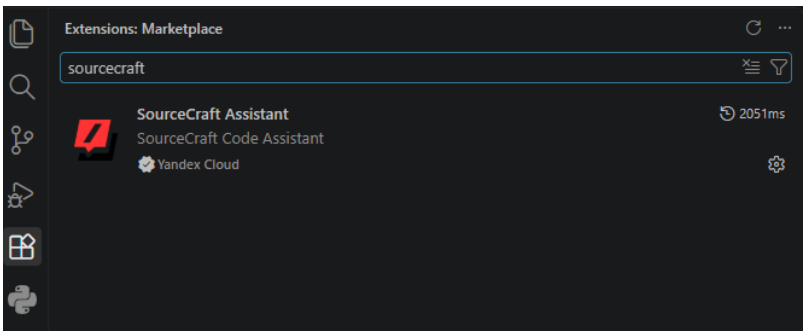


Рисунок 12 – Установка расширения Source Craft в VS Code

После установки вам будет доступен интерфейс для агентской разработки (рис.13).

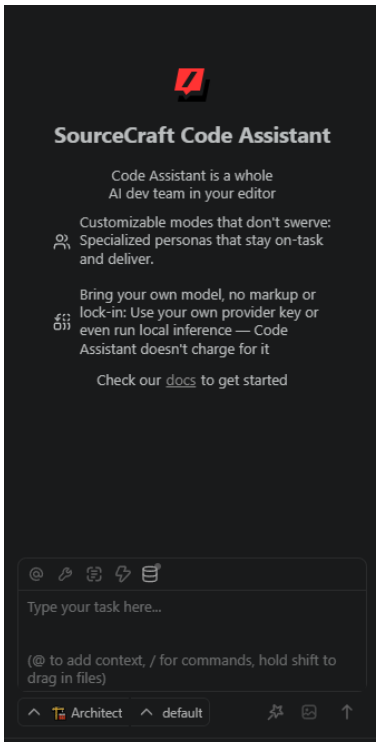
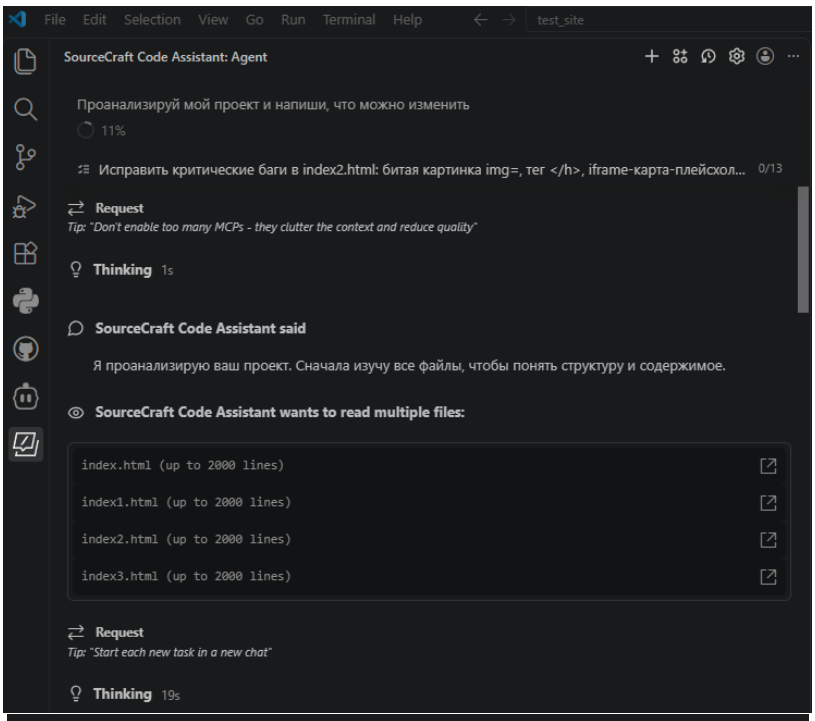


Рисунок 13 – Окно Source Craft в VS Code

Внимание: Следующий этап упражнений предлагается **не выполнять**, а изучить для ознакомления, чтобы понять, что требуется в работе, а также, чтобы сохранить доступные квоты для выполнения индивидуального задания. Таким образом, **после ознакомления вы можете приступить к выполнению своего задания**.

12. На рис 14,15 представлен результат выполнения промта «Проанализируй проект и напиши, что можно изменить». Проект должен находиться в рабочей папке.



SourceCraft Code Assistant said

Я изучил все четыре файла проекта. Это четыре независимые версии одного сайта «Кофейный котик» на чистом HTML/CSS/JS без сборки. Ниже — полный анализ с найденными проблемами и предложениями по улучшению.

Общая картина

Файл	Стиль	Иконки	Изображения	JS-интерактив
`index.html`	Apple-минимализм	эмодзи	нет	нет
`index1.html`	Playfair/Montserrat	эмодзи + Unsplash	есть	навбар, бургер, reveal
`index2.html`	Inter, тёмный hero	эмодзи + Unsplash	есть	бургер, аккордеон, скролл
`index3.html`	Segoe UI, простой	эмодзи	нет	бургер, reveal

Критические баги (index2.html)

1. `index2.html:460` — `<img="https://...">`: пропущено `src`, картинка рафа сломана.
2. `index2.html:462` — `</h>` вместо `</h4>`: некорректно закрытый тег заголовка «Медовый Раф».
3. `index2.html:583` — iframe карты с плейсхолдером `placeholder_link_hash`: карта не отобразится.
4. Внутренняя стилизация через атрибут `style` (`index.html:629`, `index.html:707`) — нарушение принципа разделения структуры и стилей.

Главная архитектурная проблема — дублирование

Один бренд реализован **4 раза**. Контент и логика раздваиваются, что ведёт к расхождениям в данных:

- **Адрес:** Кошачья 9 / Кошачья 12 / Кошачья 15 / Кофейная 7
- **Телефон:** +7 900... / +7 495 123-45-67 / +7 999...
- **Год:** 2025 / 2026
- **История:** Анна+Барсик / Алексей / 2018 Барсик
- Разные названия напитков, цены, правила бонусов

Любое изменение нужно править в 4 местах → риск рассинхрона (он уже произошёл).

Рисунок 14 – Результат работы промта в SourceCraft Code Assistant

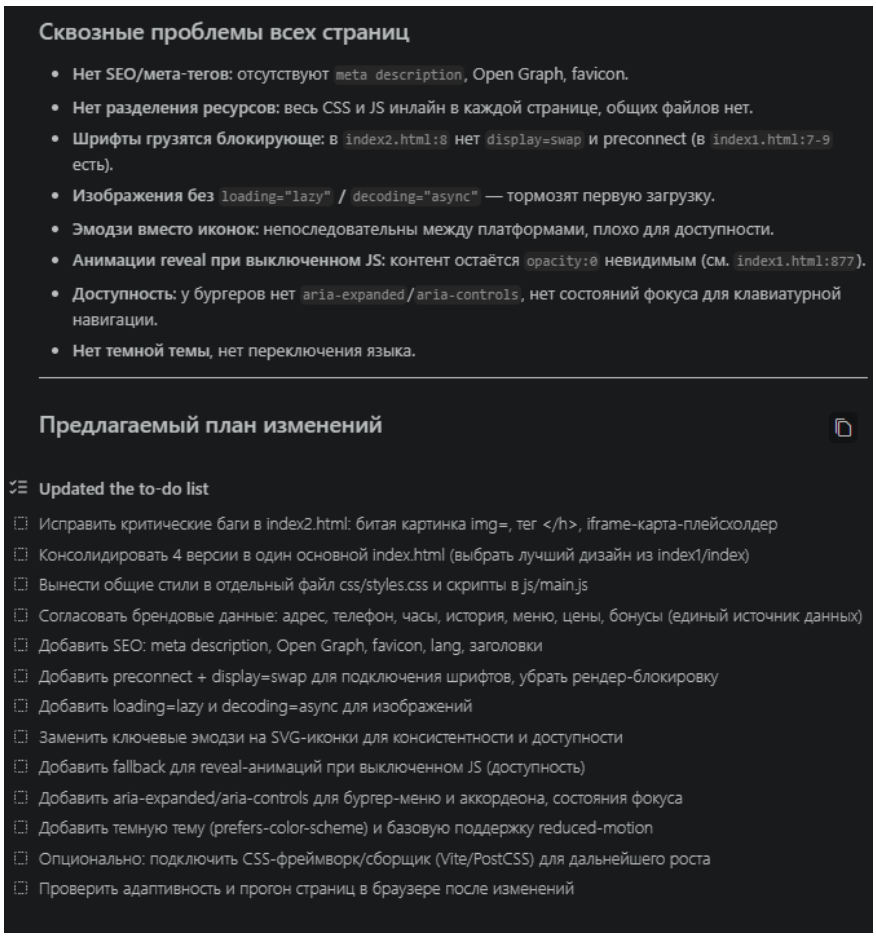


Рисунок 15 – Результат работы промта в SourceCraft Code Assistant

Видно, что основная проблема в том, что в рабочей папке сразу 4 версии сайта, созданные разными моделями, также есть проблемы с отображением некоторых элементов.

Одним из вариантов продолжения работы может быть следующий промт:

«Выбери один лучший дизайн и модернизируй его. Исправь проект, чтобы сайт выглядел более современным и был корректно разработан».

Стоит отметить, что при выполнении такого промта агент последовательно начнёт свою работу, и в какой-то момент будет запрашивать подтверждение действий (пример на рис. 16).

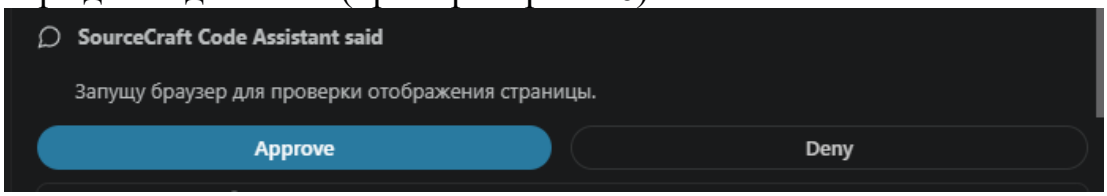


Рисунок 16 – Подтверждение запуска браузера

В результате работы код был разделен на несколько файлов: отдельно стили .css, Отдельно .js, отдельно .html. (рис.17). Такая структура считается более корректным способом создания сайтов.

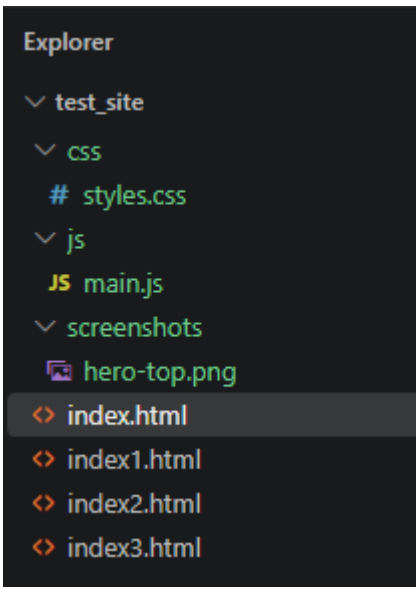


Рисунок 17 – Скриншот рабочей папки после работы агента

Дополнение:

Стоит отметить, что при работе с SourceCraft можно использовать более сложные промты, которые можно заранее сформировать (например, с помощью LLM).

Например, «*Сначала изучи 3 референсных сайта кофеен и выпиши их UX-паттерны. Затем в режиме /architect составь план реализации в Markdown. После моего утверждения переключись в /code и реализуй по шагам, запуская код после каждого компонента для проверки*». Такие формулировки позволят более точно задать последовательность действий для агента.

Проанализировав сайт, было обнаружено некорректное отображение изображения (рис.18).

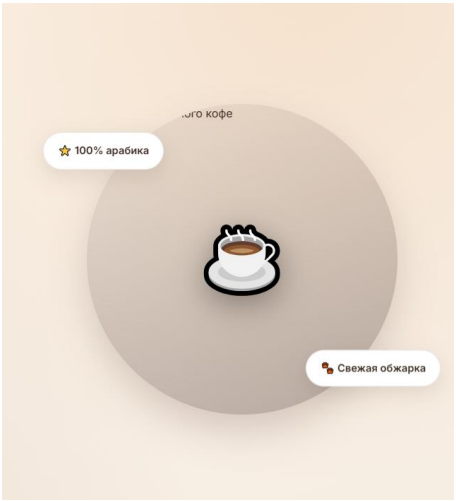


Рисунок 18 – Некорректное изображение

В связи с этим для агента появилась новая задача – исправить этот «баг» (рис.19). Видно,что агент также просит подтвердить некоторые действия и не закончит работу до подтверждения пользователя.

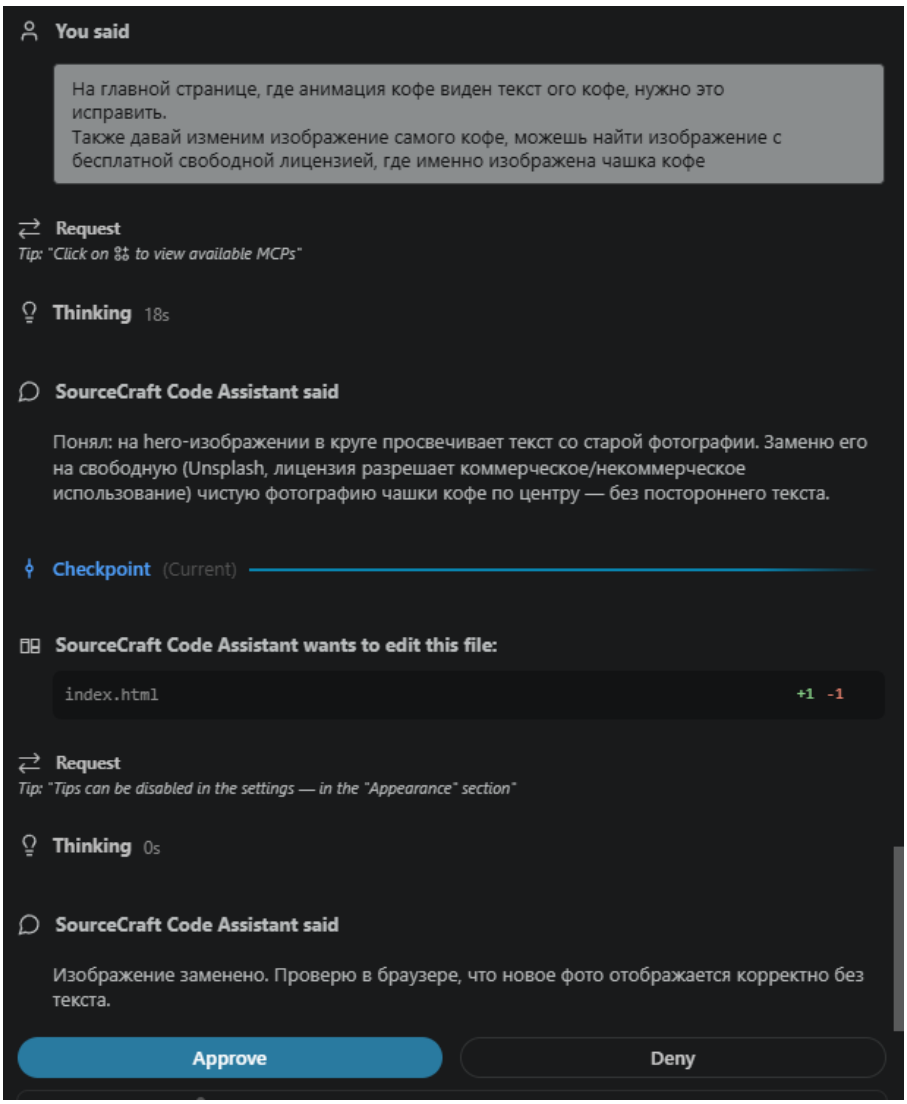


Рисунок 19 – Работа агента

Дополнение:

В настройках SourceCraft можно задать глобальные инструкции для промтов (например, *«Всегда добавляй type hinting в Python»*, *«Используй SQLAlchemy 2.0 синтаксис»*). Это избавит от необходимости повторять одни и те же требования в каждом промте.

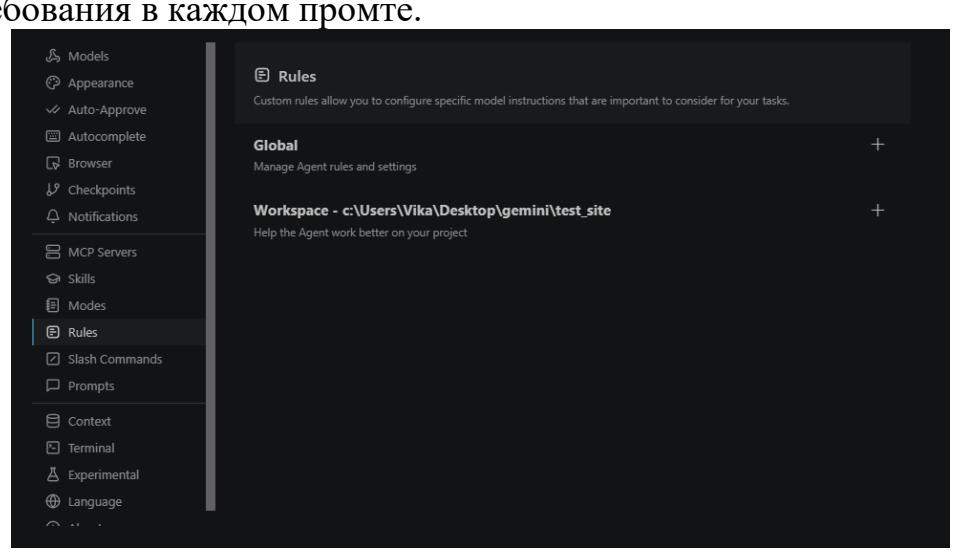


Рисунок 20 – Добавление правил

Индивидуальное задание для выполнения лабораторной работы

В рамках лабораторной работы необходимо разработать одностраничный web-сайт - лендинг на самостоятельно выбранную тему. Каждый пункт выполнения сопровождайте скриншотами и подробными пояснениями.

Сайт должен содержать **не менее 5** основных смысловых **блоков**.

Для выполнения индивидуальной части используйте пояснения из общих упражнений.

Пример:

1. Header / Навигация
2. Hero-блок
3. О компании / О проекте
4. Преимущества
5. Основные услуги / товары / предложения
6. Отзывы или дополнительные сведения
7. Контакты
8. Footer

Допускается использование другой структуры, если она соответствует выбранной предметной области, не обязательно создавать сайт для бизнеса, сайт может быть связан с хобби, обучением и т.д.

1. Выбрать предметную область (тематику), по которой вы будете создавать сайт.

Зафиксируйте:

- название проекта;
- предметную область;
- целевую аудиторию;
- основную цель сайта;
- основные действия пользователя.

Например:

Название: «Кофейный котик»

Тематика: кофейня

Целевая аудитория: студенты и молодые специалисты.

Цель: предоставить информацию по меню и привлечь посетителей.

Основное действие: перейти к меню и узнать адрес кофейни.

Отразите данный пункт в отчёте.

2. **Выбрать LLM**, с которой вы будете работать. Можно использовать любую доступную вам модель. Написать **промт для создания лендинга**. Для создания промта использовать **LLM**. Например:

«Мне нужно создать простой одностраничный лендинг для кофейни. Создай мне промт, который я смогу использовать для LLM для генерации такой лендинг-странички. Учти разные детали, такие как

работа с мобильным устройствами и компьютерами, приятный интерфейс и другие важные нюансы для современной страницы. ***Мне нужен одностраничный HTML файл с CSS.».***

На данном этапе **обязательно делайте один файл html**, как в примере.

После того, как LLM сгенерирует вам промт для создания сайта, его необходимо самостоятельно прочитать, проверить и при необходимости дополнить. Изучите этот промт, оцените, из каких частей он состоит, соответствует ли формату **R-T-F-C.**

В отчёте **отразить модель**, которую вы использовали, **промт**, который сгенерировала модель, а также ваши изменения (при наличии).

3. После этого в новом независимом диалоге (чате) используйте этот промт для создания страницы. Вы также можете найти скриншот сайта, который вам нравится, приложить этот скриншот и в промте указать «*Сделай мне лендинг в таком стиле*». Таким образом, вы приложите референс. Обязательно укажите в отчёте, если использовали референс.

4. Создайте папку проекта в VS Code, создайте файл index.html и вставьте код, который вам сгенерировала LLM. Сохраните файл и откройте сайт с помощью Live Server. Сделайте скриншот.

5. Осуществите проверку сайта:

Визуальная часть:

- читаемость текста;
- расположение элементов;
- цветовая схема;
- размеры заголовков;
- качество изображений;
- единообразие дизайна.

Функциональная часть

- работают ли ссылки;
- работают ли кнопки;
- работает ли навигация;
- корректно ли выполняется прокрутка.

Адаптивность

- на широком экране;
- на узком экране;
- в режиме мобильного устройства.

6. Выберите ещё одну модель и используйте такой же промт для создания сайта с помощью выбранной модели, сравните полученные результаты.

В отчёт приложите скриншот, опишите, результат какой модели вам понравился больше и почему, какие были отличия.

7. Теперь необходимо использовать LLM не для создания сайта, а для его критического **анализа**. Используй любую LLM и проанализируй, что можно улучшить (*SEO оптимизация, улучшение дизайна, добавление картинок*). Для этого можно вставить код в чат-бот и добавить промт.

Вариант промта:

«Проанализируй мой лендинг как опытный UX/UI-дизайнер и frontend-разработчик.

Оцени:визуальную иерархию; цветовую схему; типографику;навигацию; структуру страницы; CTA-элементы; адаптивность; качество текста; потенциальные проблемы UX;SEO;визуальную современность.

Для каждого недостатка предложи конкретное исправление.

Не переписывай весь сайт. Сначала составь список приоритетных улучшений.»

Приложи скриншот с результатом анализа (можно фрагмент), сделай выводы, насколько хорошо модель смогла оценить сайт.

8. Теперь необходимо использовать ИИ-агента. В данной работе предлагается использовать https://sourcecraft.dev через расширение в VS Code (как показано в упражнении). *Если у тебя есть доступ к другим агентам, то по согласованию с преподавателем практики можно использовать другой доступный ресурс.*

В отчёте укажи, с каким агентом ты будешь работать.

9. Используя ИИ-агента для разработки внести изменения в проект. Самостоятельно или с помощью LLM сформулируйте две небольшие задачи для агента.

Обратите внимание, что не нужно в данной работе создавать идеальный сайт и исправлять все недочеты. Оставьте код в одном файле, а также оставьте некоторые неточности, чтобы исправить их в следующей лабораторной работе. Покажите в данной работе два улучшения сайта.

Можно непосредственно в интерфейсе агента выполнить следующее действие *«Проанализируй текущий проект, структуру файлов и разметку index.html. Составь перечень технических проблем и архитектурный план оптимизации.»*.

Во время работы агента наблюдай за его действиями, следите, какие файлы он читает, изменяет, при необходимости давайте ответы.

В отчёте опишите ваши наблюдения, насколько хорошо были выполнены задачи, а также приложите скриншот из чата с агентом и скриншот изменений.

Структура отчёта:

1. Титульный лист по ГОСТ
2. Цель работы
3. Постановка задачи (тематика сайта, цель, целевая аудитория, технологии).
4. Исходный промт для генерации первоначального сайта и название LLM. Оценка промта.
5. Скриншот сайта и описание результата.
6. Результат анализа сайта с помощью LLM (промт для LLM, название LLM, выявленные проблемы и рекомендации, ваши выводы).
7. Указать название используемого агента, задачу, какие изменения были выполнены.
8. Скриншот сайта с изменениями.
9. Расширенный вывод в формате эссе с описанием преимуществ и ограничений использования агентов и LLM.

Обратите внимание, что вы должны отразить весь ход выполнения работы в отчёте, со скриншотами и подробными пояснениями к ним.

Контрольные вопросы

1. Что такое искусственный интеллект?
2. Что такое генеративный искусственный интеллект?
3. Что такое LLM?
4. Для чего используются большие языковые модели?
5. Что такое токен?
6. Что такое контекст LLM?
7. Что такое контекстное окно?
8. Почему большой программный код может быстро занимать контекстное окно?
9. Что такое чат-бот?
10. Чем чат-бот отличается от LLM?
11. Что такое промпт?
12. Почему качество промпта влияет на результат работы LLM?
13. Что означает принцип R-T-F-C?
14. Что такое Role?
15. Что такое Context?
16. Что такое Task?
17. Что такое Format?
18. Что такое Constraints?
19. Что такое AI-агент?
20. Чем AI-агент отличается от обычного чат-бота?
21. Какие инструменты может использовать AI-агент?
22. Может ли AI-агент самостоятельно изменять файлы проекта?
23. Почему AI-агент может быть эффективнее чат-бота при разработке программного обеспечения?
24. Какие риски возникают при использовании AI-агентов?
25. Почему действия AI-агента необходимо контролировать?
26. В каких случаях достаточно обычной LLM?
27. В каких случаях целесообразно использовать AI-агента?
28. Что такое итеративная работа с LLM?
29. Почему нельзя полностью доверять результату, полученному от ИИ?
30. Как можно проверить качество Web-сайта, созданного ИИ?